

Two feeds of a set-monad on containers: the entwining λ and the arrow κ

MacBeth
Kodamai

4 August 2026

Abstract

A single monad M on **Set** acts on the category **Cont** of containers along *two* axes. Along shapes it gives the Ahman–Bauer effect monad T_M ; along positions (which are contravariant) it gives a coeffect comonad G_M . This note records, for a reader fluent in monads, comonads and distributive laws, the definitions leading up to the two mixed distributive laws relating these feeds, and their precise statements: the *entwining* $\lambda: T_M G_M \Rightarrow G_M T_M$, which exists for every set-monad M (the bialgebra / Plotkin–Turi face), and the *arrow compositor* $\kappa: G_M T_M \Rightarrow T_M G_M$, which exists if and only if M is non-branching, equivalently $M \cong E + A \times (-)$. Definitions and statements only; proofs are cited to the author’s notes.

1 Containers and the extension functor

Definition 1 (Container). A *container* is a pair (S, P) with $S \in \mathbf{Set}$ a set of *shapes* and $P: S \rightarrow \mathbf{Set}$ a family of *positions*. A *morphism* $(S, P) \rightarrow (T, Q)$ is a pair (u, f) where $u: S \rightarrow T$ runs *forward* on shapes and, for each $s \in S$, $f_s: Q(us) \rightarrow P(s)$ runs *backward* on positions. Composition is forward on shapes and backward (in the opposite order) on positions; the identity is $(\text{id}_S, \{\text{id}_{P_s}\})$. Containers and their morphisms form a category **Cont**.

The single fact that positions are *contravariant* is the hinge of everything below.

Definition 2 (Extension). The *extension* functor sends a container to the polynomial endofunctor it presents:

$$\llbracket - \rrbracket: \mathbf{Cont} \longrightarrow [\mathbf{Set}, \mathbf{Set}], \quad \llbracket S, P \rrbracket(X) = \sum_{s \in S} X^{P(s)}.$$

It is fully faithful, with image the polynomial endofunctors; we freely identify a container with the functor it presents. A morphism (u, f) acts by $(s, g: Ps \rightarrow X) \mapsto (us, g \circ f_s)$.

Finite products in **Cont** are computed by $(S, P) \times (T, Q) = (S \times T, (s, t) \mapsto P(s) \sqcup Q(t))$, matching $\llbracket p \times q \rrbracket = \llbracket p \rrbracket \times \llbracket q \rrbracket$; this is the cartesian tensor used in §5.

Set-monads with support. Throughout, $M = (M, \eta, \mu)$ is a set-monad presented as a polynomial functor

$$MX = \sum_{s \in M1} X^{\text{ar}(s)}, \quad \text{ar}: M1 \rightarrow \mathbf{Set},$$

so each $m \in MX$ has a *support* of *leaves* $b \in \text{lv}(m)$ carrying labels $x_b \in X$. Say M is *non-branching* if $|\text{ar}(s)| \leq 1$ for every shape (equivalently every m has $|\text{lv}(m)| \leq 1$), and *branching* otherwise.

Flagship non-branching monads: **Maybe** = $1 + (-)$, exception $E + (-)$, **Writer** $_N = N \times (-)$, **Id**.
Flagship branching monads: finite powerset **Pf** and **List**.

To feed M into positions we use the Ahman–Bauer $\prod$ -*cointerpretation* predicate lifting $(-)^*$, which turns a family $P: S \rightarrow \mathbf{Set}$ into $P^*: MS \rightarrow \mathbf{Set}$ by

$$P^*(m) = \prod_{b \in \text{lv}(m)} P(x_b),$$

the universal (product-based) lifting [1, §6]. Its unit datum is the projection out of the singleton product $P^*(\eta_S s) = \prod_{\{*\}} P s = P s$; its multiplication datum reindexes a product along the leaf-map of μ^M . We call such an M a $\prod$ -*cointerpretation monad*; this is the standing hypothesis, and it covers all the flagship examples. (*Non-branching* means $\text{arity} \leq 1$; this is *not* Kock-affineness $M1 \cong 1$, which would force $M = \text{Id}$.)

2 The effect monad T_M

Shapes are covariant, so M lifts along them to a *monad* on **Cont**.

Definition 3 (Effect monad, [1, Thm. 6.3]). For a $\prod$ -cointerpretation monad M ,

$$T_M(S, P) = (MS, P^*), \quad P^*(m) = \prod_{b \in \text{lv}(m)} P(x_b).$$

The unit η^T covers η^M on shapes and is backward the singleton projection; the multiplication μ^T covers μ^M and is backward the product-reindexing along the leaf-map of μ^M . On extensions, $\llbracket T_M(S, P) \rrbracket(X) = \sum_{m \in MS} X^{P^*(m)}$: an M -structured computation over the operations of (S, P) , whose positions are the tuples of operands, one per leaf.

Here $P^*(m) = \prod_{b \in \text{lv}(m)} P(x_b)$ is the $\prod$ -*cointerpretation*: the product, over the leaves of m , of the position sets at the labels those leaves carry. Because shapes are covariant, the monad structure of M transports directly, and T_M is a monad on **Cont** — the *effect* feed.

3 The coeffect comonad G_M

Positions are contravariant, so the same M lifts along them to a *comonad*.

Definition 4 (Coeffect comonad, transfer). For any set-monad M ,

$$G_M(S, P) = (S, M \circ P), \quad G_M(u, f) = (u, \{M(f_s)\}_s).$$

The counit ε is the identity on shapes and backward $\eta_{P_s}^M: P s \rightarrow M P s$; the comultiplication δ is the identity on shapes and backward $\mu_{P_s}^M: M M P s \rightarrow M P s$. On extensions, $\llbracket G_M(S, P) \rrbracket(X) = \sum_s X^{M(Ps)}$: each operand slot acquires an M -structured context.

Proposition 5 ([7, monad–comonad transfer note]). $(G_M, \varepsilon, \delta)$ is a comonad on **Cont**. Its three comonad laws are exactly M 's three monad laws pulled back through position-contravariance: counit-left $\Leftrightarrow M$ right-unit, counit-right $\Leftrightarrow M$ left-unit, coassociativity $\Leftrightarrow M$ associativity.

Conceptually, the fibre of the shape-projection $\mathbf{Cont} \rightarrow \mathbf{Set}$ over S is $(\mathbf{Set}^{\text{op}})^S$, and G_M acts fibre-wise as $(M^{\text{op}})_*$; a monad on $\mathbf{Set}$ is a comonad on $\mathbf{Set}^{\text{op}}$, so “positions are contravariant” is precisely the $(-)^{\text{op}}$ that turns M into a comonad. Equivalently $G_M(S, P) = \{M/(S, P)\} = \text{Lan}_{(S, P)} M$, the left coclosure of the substitution product with M in the numerator. This is the *coeffect* feed.

The two liftings are welded by the fibrewise $(-)^{\text{op}}$ that makes a container a container: apply M where variance runs forward and monads stay monads; apply it where variance runs backward and monads become comonads. There is no third axis and no choice in the outcome.

4 The entwining $\lambda: T_M G_M \Rightarrow G_M T_M$

Recall (e.g. [2]) that a *mixed distributive law* (entwining) of a comonad G over a monad T on $\mathcal{C}$ is a natural transformation $\lambda: TG \Rightarrow GT$ satisfying four axioms — two for the unit and multiplication of T , two for the counit and comultiplication of G . Such a λ is exactly the datum lifting G to a comonad on $\text{Alg}(T)$ and T to a monad on $\text{Coalg}(G)$; a compatible algebra-coalgebra is a λ -*bialgebra* in the sense of Turi–Plotkin [3].

Both $T_M G_M$ and $G_M T_M$ are the identity on shapes over MS , so it suffices to read positions at a fixed shape $m \in MS$. Writing $Z_b := P(x_b)$ for the position set at leaf $b \in \text{lv}(m)$:

functor	positions at m
$G_M T_M$	$M(P^\star m) = M(\prod_b Z_b)$
$T_M G_M$	$(M \circ P)^\star(m) = \prod_b M(Z_b)$

Between these there is exactly one canonical map, the *oplax product-comparison*

$$\text{st}_{(Z_b)}: M(\prod_b Z_b) \longrightarrow \prod_b M(Z_b), \quad w \longmapsto (M(\pi_b) w)_b,$$

built from M and the product projections alone. It exists for *every* functor M (it is the oplax monoidal structure of M for $(\mathbf{Set}, \times)$); no algebra structure on M is used. Since container morphisms run their position map backward, st is the backward part of a $\mathbf{Cont}$ -morphism the *other* way, giving a 2-cell $\lambda: T_M G_M \Rightarrow G_M T_M$ (forward id_{MS} , backward st). The orientation is forced: the opposite direction would need the reverse map $\prod_b M(Z_b) \rightarrow M(\prod_b Z_b)$, which is not canonical.

Theorem 6 (The two feeds always entwine, [8, monad–comonad entwining note]). *Let M be a $\prod$ -cointerpretation set-monad. The 2-cell $\lambda: T_M G_M \Rightarrow G_M T_M$ with backward map st is natural and satisfies the four entwining axioms, for every such M (branching or not). Consequently G_M lifts to a comonad on $\text{Alg}(T_M)$ and T_M lifts to a monad on $\text{Coalg}(G_M)$: a Turi–Plotkin λ -bialgebra.*

The content is a single statement: *M oplax-preserves products, coherently with its unit and multiplication, transported onto positions.* The counit and comultiplication axioms are the naturality of η^M and μ^M at the projections π_b ; the unit axiom is the collapse of a length-one product; the multiplication axiom is the naturality of st with respect to the product-reindexing that is the multiplication datum of $P^\star$. In particular the bialgebra face *keeps* the branching monads: **Pf**, **List** and probabilistic branching all carry the λ -bialgebra exactly as **Maybe** does. The denotational unification of effects and coeffects on containers is unconditional.

5 The arrow $\kappa: G_M T_M \Rightarrow T_M G_M$

The reverse orientation reads as a *composition rule*. An *effect-coeffect arrow* $p \rightsquigarrow q$ is a container morphism $G_M p \rightarrow T_M q$: it reads its input through the coeffect G_M and delivers its output through the effect T_M — the container analogue of a $\text{coKleisli-of-}G\text{-then-Kleisli-of-}T$ program $GA \rightarrow TB$.

Definition 7 (The effect-coeffect arrows). For containers p, q set $\text{Arr}_M(p, q) = \text{Cont}(G_M p, T_M q)$. The intended identity $p \rightsquigarrow p$ is $\eta_p^T \circ \varepsilon_p$, and the intended composite of $f: G_M p \rightarrow T_M q$ and $g: G_M q \rightarrow T_M r$ is the biKleisli composite

$$G_M p \xrightarrow{\delta_p} G_M G_M p \xrightarrow{G_M f} G_M T_M q \xrightarrow{\kappa_q} T_M G_M q \xrightarrow{T_M g} T_M T_M r \xrightarrow{\mu_r^T} T_M r.$$

The middle step is a natural transformation $\kappa: G_M T_M \Rightarrow T_M G_M$: it commutes the effect out through the coeffect. This is the *reverse* of λ ; on positions its backward map is the *lax* product-comparison $\prod_b M(Z_b) \rightarrow M(\prod_b Z_b)$, which requires extra structure on M (a commutative M supplies the cartesian one) and, even when it exists, need not be a distributive law.

Theorem 8 (Existence of the arrow category, [9, effect-coeffect arrows note]). *For a $\prod$ -cointerpretation set-monad M the following are equivalent.*

1. *The arrows $\text{Arr}_M(p, q) = \text{Cont}(G_M p, T_M q)$ form a category with identity $\eta^T \circ \varepsilon$ and the biKleisli composition of Definition 7.*
2. *There is a natural $\kappa: G_M T_M \Rightarrow T_M G_M$ satisfying the four mixed-distributive-law axioms E1'–E4'.*
3. *M is non-branching.*

The unit laws correspond to E1', E3' and hold for every M ; the sole obstructed axiom is associativity, corresponding to E2' (the multiplication- T axiom).

Example 9 (Branching breaks associativity). For $M = \text{Pf}$ the unit laws hold and only associativity fails. At a double shape $\{\{a, b\}, \{a\}\} \in \text{Pf Pf } S$ the two associations of a composite return, on one side, a *union of products* and, on the other, the full *product of unions*: $\mu^M = \bigcup$ merges the leaf a shared by $\{a, b\}$ and $\{a\}$, and the lax map cannot enforce that the two choices at that shared leaf agree. The obstruction is *branching*, not non-commutativity (Pf is commutative and still fails); a shape with ≥ 2 positions is needed for $\bigcup$ to overlap. When arity is ≤ 1 no two leaves are ever shared, μ^M merges none, E2' holds trivially, and both orientations are entwinings at once.

For non-branching M the category Arr_M is moreover a *Hughes arrow*, equivalently a *Freyd category* over the cartesian base $(\text{Cont}, \times)$: with the pure functor $\text{arr}(\varphi) = \eta_q^T \circ \varphi \circ \varepsilon_p$, the costrength of G_M (which exists for all M), and the tensorial strength of T_M (which exists *iff* M is non-branching), the data satisfy the arrow laws. So branching disables the arrow twice over: through E2' (associativity, by leaf merging) and, independently, through the absence of a natural strength for T_M (by leaf symmetry).

The non-branching monads are named exactly [9]: a cartesian $\prod$ -cointerpretation M is non-branching *iff* $M \cong E + A \times (-)$ as a functor, *iff* M is a *writer-with-absorbing-exceptions* monad — $(A, \cdot, e)$ a monoid, $(E, \odot)$ a left A -set, writer unit $x \mapsto (e, x)$, the log A acting on the exceptions E . The arrow category exists throughout this class.

6 The two faces

One monad M , two feeds T_M and G_M , and two comparison maps between their composites, behaving oppositely.

	$\lambda: T_M G_M \Rightarrow G_M T_M$ (oplax st)	$\kappa: G_M T_M \Rightarrow T_M G_M$ (lax comparison)
what it is	entwining / Turi–Plotkin bialgebra	compositor of $\text{Arr}_M(p, q) = \text{Cont}(G_M p, T_M q)$
when	every M	iff M non-branching $\cong E + A \times (-)$
face	denotational (semantics)	operational (arrow / Freyd)

The oplax comparison st runs $T_M G_M \Rightarrow G_M T_M$ and is always a law: the two feeds of *any* monad entwine, so the effect comonad acts on effect-algebras and the effect monad on coeffect-coalgebras, nondeterminism included. The lax comparison runs the other way, $G_M T_M \Rightarrow T_M G_M$, and is the compositor that lets one *sequentially compose* effect–coeffect programs into a category; it is a distributive law exactly when M does not branch. For arity ≤ 1 the two coincide (up to coherent unit padding), giving the clean case where operational and denotational agree. Branching obstructs the *orientation* of the effect–coeffect interaction, not the interaction itself: the bialgebra survives, the arrow does not.

References

- [1] D. Ahman and A. Bauer, *Comodules and contextads over containers*, arXiv:2409.17664, 2024. (Effect monad T_M : Thm. 6.3; the Π -cointerpretation predicate lifting: §6.)
- [2] J. Power and H. Watanabe, *Combining a monad and a comonad*, Theoretical Computer Science **280** (2002), 137–162.
- [3] D. Turi and G. Plotkin, *Towards a mathematical operational semantics*, Proc. LICS 1997, 280–291.
- [4] J. Hughes, *Generalising monads to arrows*, Science of Computer Programming **37** (2000), 67–111.
- [5] R. Atkey, *What is a categorical model of arrows?*, Electronic Notes in Theoretical Computer Science **229** (2011), 19–37.
- [6] T. Uustalu and V. Vene, *Comonadic notions of computation*, Electronic Notes in Theoretical Computer Science **203** (2008), 263–284.
- [7] MacBeth, *The monad–comonad transfer on containers* (proof note, 25 July 2026). $G_M(S, P) = (S, M \circ P)$ is a comonad; proved and Lean-verified.
- [8] MacBeth, *The two container-liftings of a set-monad entwine — in exactly one direction* (proof note, 27 July 2026). The law λ and its four axioms.
- [9] MacBeth, *Effect-coeffect arrows on Cont : the compositor is the reverse entwining, and it exists iff M does not branch* (proof note, 29 July 2026). Theorem 8 and the $E + A \times (-)$ classification.